ADR 0010 — Telemetry

Status: Proposed (Marv to confirm) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0002, ADR 0006, ADR 0007, src/baymax/telemetry/logger.py

---

Context

BAYMAX needs telemetry that:

1. Records every event in the agent loop with enough context to reconstruct what happened after the fact
2. Never leaks secrets (API keys, auth tokens, OAuth credentials) into trace files, log files, or stderr
3. Correlates events within a single request across modules (agent core → inference backend → tool adapter)
4. Is consumed both by Ronin's eval harness (for failure analysis) and by developer eyeballs (for debugging during dev)
5. Has zero new infrastructure requirements in v1 — same posture as state store (ADR 0007) and trace format (ADR 0006)

The constraint set is shaped by:

- ADR 0002 §6: environment-based auth → tokens must never appear in traces
- ADR 0006: ndjson trace format — telemetry emits ndjson events
- Single-process v1 — no distributed tracing requirement yet

The choice surface is "which library writes the JSON" — not whether to use JSON. The candidates:

- `structlog` — structured logging library; emits dict events; pluggable processors
- Python stdlib `logging` with a custom JSON formatter — works but ergonomically rough
- OpenTelemetry SDK directly — full distributed-tracing semantics, overkill for v1
- Roll our own — re-invents the wheel for no benefit

Decision

`structlog` **for v1**, configured to:

- Emit ndjson events (one event per line, matching ADR 0006)
- Run a redaction processor on every event before serialisation
- Inject a `trace_id` context variable so events across modules within one request correlate
- Write to `traces/<run_id>/<task_id>.jsonl` (per ADR 0006 path)

Redaction processor

Any event field whose key matches one of the following case-insensitive patterns is replaced with `[REDACTED]` before the event is written:

- `auth*`
- `token*`
- `key*`
- `password*`
- `secret*`
- `authorization*`
- `bearer*`

The processor runs as the LAST step in structlog's chain, after all event construction. This means any code path that accidentally includes a credential in an event payload gets caught at the chokepoint, not at the call site.

Trace correlation

`trace_id` is bound via `structlog.contextvars.bind_contextvars(trace_id=...)` at the start of each request. All subsequent events emitted in that request (across modules) automatically include the trace_id. When the state store ships in v2 (ADR 0007 — deferred as of 2026-06-23), it will join traces to tasks via this id. In v1 the join is done by grep/`jq` against the `traces.jsonl` file.

Event shape

Matches ADR 0006 event shape exactly — telemetry is the writer; ADR 0006 is the schema.

OpenTelemetry deferral

OTel format remains deferred to v2 pending Ronin discussion (per RONIN_SYNC.txt Tier 1 #1 and ADR 0006). When OTel migration happens, structlog's processor chain swaps the final JSON formatter for an OTel exporter — the rest of the emission code is unaffected.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months.]

- **WHY structlog over stdlib** `logging`: stdlib's model is "message + optional extra dict"; passing structured event payloads naturally is awkward. Custom formatters get ugly, and filters run BEFORE message construction in some paths, making redaction unreliable. Structlog is built specifically for structured events and processor pipelines.
- **WHY structlog over rolling our own logger**: reimplements structlog badly. The only reason to roll our own would be if structlog had a limitation we hit — it doesn't. Mature, well-tested, right shape.
- **WHY redaction at the LAST step in the processor chain, not at the call site**: single chokepoint that can't be bypassed by a forgotten log statement. If someone next week adds a new credential field to an

event payload, it gets caught at emission — nobody has to remember to update N call sites. Redaction at the call site scales badly; every new field is a new opportunity to leak.

- **WHY contextvars over passing `trace_id` through call signatures**: correlation is a cross-cutting concern, not an application-logic concern. Threading `trace_id` through every function signature pollutes the code with infrastructure detail and makes refactoring painful. Contextvars solve the correlation problem WITHOUT polluting call signatures — cleaner separation.
- **WHY defer OTel to v2** (aligned with ADR 0006): the SDK is 30+ MB of dependency for capability we don't use in single-process v1. When OTel migration happens, structlog's processor chain swaps the final formatter for an OTel exporter — the emission sites are unaffected. Migration is bounded by design.
- **WHY the specific redaction pattern list (`auth*`, `token*`, `key*`, `password*`, `secret*`, `authorization*`, `bearer*`)**: covers the credential fields that show up in adapter payloads and API responses. Not exhaustive — the list is reviewed quarterly as new adapters land (documented as an open question). Defensible baseline, not a claim of completeness.

Alternatives considered

Alternative A — Python stdlib `logging` with a custom JSON formatter

Use `logging.getLogger(__name__)`, configure a custom formatter that emits JSON, add a filter for redaction.

why rejected — stdlib logging's data model is "message + optional extra dict"; you can't naturally pass structured event payloads. Custom formatters get ugly fast. Filters run BEFORE message construction in some paths, making redaction unreliable. Structlog was built specifically for this use case.

Alternative B — OpenTelemetry SDK directly

Adopt OTel's tracer / span / attribute model; use the OTel SDK's exporters and instrumentation libraries.

why rejected — OTel buys distributed-trace correlation we don't need yet (single-process v1). The SDK is heavy (30+ MB dependencies). The semantic conventions assume a span-based mental model that doesn't perfectly match our event-stream model. ADR 0006 already deferred OTel to v2; this ADR follows that decision.

Alternative C — Roll our own logger

Write a small custom logger class that emits ndjson, handles redaction, manages context.

why rejected — re-implements structlog, badly. Structlog is mature, well-tested, has the right shape. The only reason to roll our own would be if structlog had some specific limitation we hit — it doesn't.

Alternative D — `loguru`

A popular alternative to structlog with a simpler API.

why rejected — loguru's redaction story is weaker (you have to filter messages by regex, no native field-level processor chain). For "must redact every credential field every time" requirements, structlog's processor chain is the right primitive.

Consequences

Positive

- Redaction is a single chokepoint that can't be bypassed by a forgotten log statement
- Adding fields to events is trivial (just pass them as kwargs to the logger call)
- Trace correlation across modules is implicit (no manual trace_id threading through call signatures)
- Migration to OTel in v2 is local to the structlog processor chain — emission sites are unaffected
- Cost per event is sub-microsecond on modern Macs (benchmark Phase 1 confirms)

Negative

- structlog adds a dependency and a small learning curve for new contributors
- The processor chain is the kind of "implicit configuration" that surprises people the first time they hit it — needs to be documented in `src/baymax/telemetry/README.md`
- Contextvars don't propagate across `asyncio.create_task` boundaries unless you propagate manually; need a wrapper

Neutral

- Redaction patterns are listed in code; any new credential category not on the list could leak. Mitigation: review the redaction list quarterly as new adapters are added

---

Open questions / follow-ups

- **Asyncio context propagation**: confirm that contextvars survive `asyncio.gather` boundaries in our specific call paths; test with a tool dispatched via `gather` in Phase 1.
- **Per-module log levels**: not needed yet (everything emits at INFO). Add per-module filtering when log volume becomes a problem.
- **Sampling**: not in v1 (every event is emitted). Add per-event-category sampling rates when volume is a problem at Phase 3.
- **Sensitive payload redaction beyond credentials**: e.g., should the body of a user's email draft be redacted in traces? Not in v1 (the agent has to work with the content). Worth a v2 discussion when real adapters arrive and traces start including real personal data.
- **OTel migration trigger**: tracked alongside ADR 0006 in DEFERRED.md.

References

- structlog docs: https://www.structlog.org
- structlog processor chain pattern: https://www.structlog.org/en/stable/processors.html
- Python contextvars (PEP 567): https://peps.python.org/pep-0567/
- ADR 0002 §6 — env-based auth requirement that drives redaction

- ADR 0006 — ndjson event format this ADR emits
- ADR 0007 — state store this ADR's trace_ids join into
- RONIN_SYNC.txt Tier 1 #1 — OTel-vs-custom discussion

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.